

Examen 2 (20%)

Fecha: Octubre de 2026

Documento	Apellidos y Nombres	Nota
-----------	---------------------	------

Sistema de habilidades a evaluar: Aplicar los conceptos básicos de la lógica de programación y la orientación a objetos en el desarrollo de una aplicación con interfaz gráfica de usuario basada en el lenguaje Java y un IDE adecuado.

NOTAS:

- Esta evaluación representa en total una calificación de 20% correspondiente al Examen Parcial

Elaborar la respectiva aplicación en un *Java* para los siguientes enunciados:

1. Una empresa de envíos ofrece 3 medios para estos: Terrestre, Aéreo y Fluvial. Todos los envíos manejan la siguiente información común:
 - Cliente (nombre del remitente)
 - Código del envío (único)
 - Peso en kilogramos
 - Distancia en kilómetros

El cálculo de la tarifa tendrá implementaciones diferentes según el medio de transporte:

Medio	Tarifa base por Km	Recargo por Kg
Terrestre	\$ 1500	\$ 2000
Aéreo	\$ 5000	\$ 4000
Marítimo	\$ 800	\$ 1000

Realizar un aplicativo que permita:

- **Agregar envío:** solicitando al usuario el tipo y los demás datos
- **Retirar envío:** solicitar el código del envío y eliminar el envío correspondiente. Si el código no existe, informar al usuario.
- **Listar los envíos:** mostrar para cada envío:
 - Código
 - Cliente
 - Tipo de envío
 - Peso
 - Distancia
 - Tarifa calculada

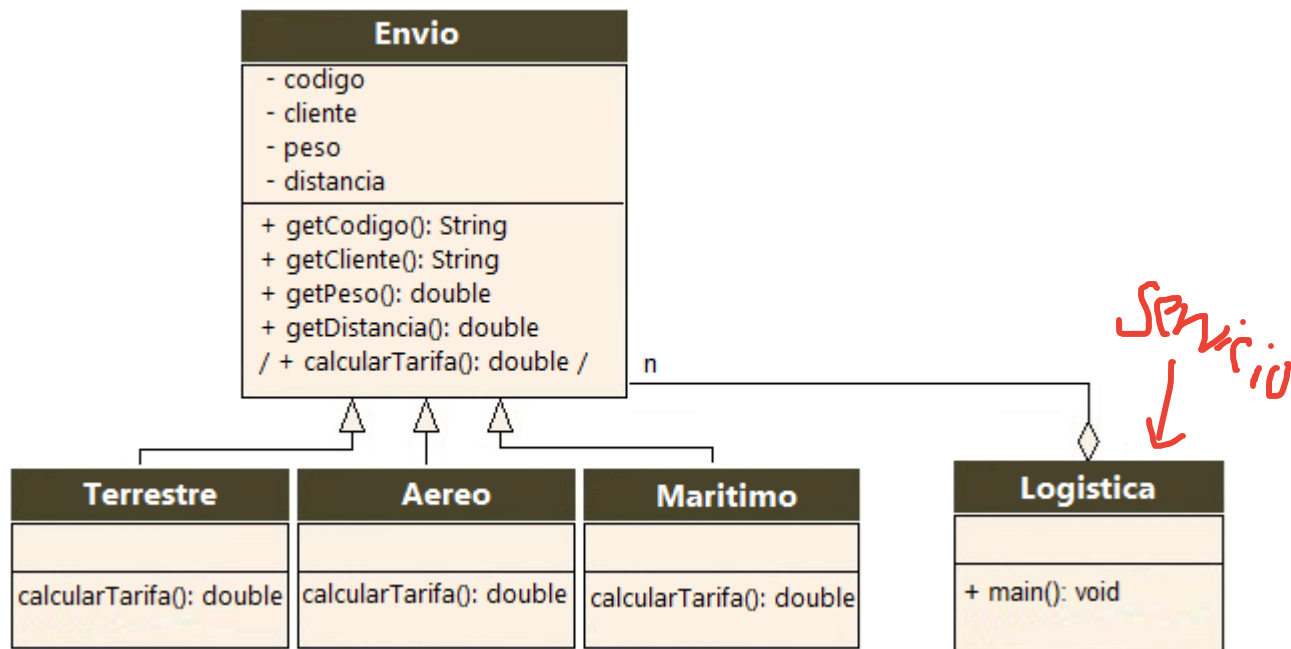


Operador Logístico

Número: 10004 Tipo: Marítimo
Cliente: Cafe de Colombi Distancia en Km: 3000
Peso: 3000 Guardar Cancelar

Tipo	Código	Cliente	Peso	Distancia	Costo
Terrestre	10001	Polimeros Col...	1200.0	400.0	3000000.0
Terrestre	10002	Textiles Pepalfa	500.0	600.0	1900000.0
Aéreo	10003	Flores Colombi...	1500.0	2000.0	16000050

Se sugiere el siguiente diagrama de clases



El sistema debe ser diseñado aplicando principios de Programación Orientada a Objetos (POO) y código limpio:

Polimorfismo

La tarifa deberá calcularse mediante polimorfismo, invocando el método correspondiente sobre la referencia *Envio*.

Arquitectura

La aplicación debe organizarse utilizando una estructura **MVC + Service**, de manera que:

- **Vista:** gestione exclusivamente la interfaz gráfica.
- **Controlador:** gestione los eventos de la interfaz y coordine las operaciones.
- **Modelo:** represente los envíos y sus diferentes tipos.
- **Servicio:** gestione las operaciones sobre la colección de envíos.
- **Factory:** sea responsable de la creación de los diferentes tipos de *Envío*.

Principios SOLID

- S (Single Responsibility): Cada clase debe tener una única responsabilidad.
- (Open/Closed): El sistema debe permitir agregar nuevos tipos de envío sin modificar código existente.
- L (Liskov Substitution): Cada subtipo de envío debe poder usarse como un envío genérico sin alterar el comportamiento esperado.
- I (Interface Segregation): Las interfaces deben ser específicas a las necesidades (si se crean).
- D (Dependency Inversion): El código de alto nivel no debe depender de implementaciones concretas

Buenas prácticas

- Aplicar nombres descriptivos y convenciones Java.
- Evitar duplicación de código (DRY).
- Implementar encapsulamiento adecuado (atributos private, getters/setters).
- Utilizar colecciones genéricas (por ejemplo, List<Envío>).
- Utilizar polimorfismo para el cálculo de la tarifa.
- Dividir la lógica de negocio y la interfaz de usuario en clases separadas

Patrón de diseño Factory Method

La creación de los objetos *Envío* debe realizarse aplicando el patrón de diseño **Factory Method**.

- Crear una clase abstracta que defina el método para crear un objeto *Envío*
- Implementar una fábrica concreta para cada tipo de envío
- La lógica de creación de los objetos *Terrestre*, *Aereo* y *Maritimo* no debe estar directamente en el controlador ni en la interfaz gráfica ni en el servicio
- El código cliente debe trabajar con la abstracción *Envío*, no con las clases concretas cuando no sea necesario

Extensibilidad: suponga que posteriormente se requiere incorporar un nuevo medio de transporte denominado Express. La solución debe permitir incorporar este nuevo tipo mediante una nueva clase de envío y su correspondiente fábrica, evitando modificar la lógica existente de las fábricas de los otros tipos.